APCS 程式設計系列 · U07 · 實作中級 · Python版

函式與遞迴 詳解

函式與回傳值 · 傳值與傳參考 · 區域與全域變數 · 遞迴三要素 · 遞迴追蹤 · 呼叫次數爆炸 · 河內塔與分治

看這個單元的講解影片（YouTube）
【APCS Python】U07 函式與遞迴｜函式、遞迴追蹤、河內塔｜9 分鐘重點講解
錯比較多？回去看講解影片，手機掃描 QR code 或點這裡 https://youtu.be/Mxc9zDbxt1A

Part A 答案速查

1	2	3	4	5	6	7	8	9	10
B	C	D	D	A	B	B	C	A	D
11	**12**	**13**	**14**						
A	C	C	D						

Part A 程式識讀詳解

第 1 題 答案：(B)
- 先算裡面的 f(1, 2) = 1 × 2 + 2 = 4。
- 再算 f(4, 3) = 4 × 2 + 3 = 11。

第 2 題 答案：(C)
- 整數 x 傳進去的是值，函式裡改的是自己的 x，外面的 x 還是 1。
- list 是同一個物件，a[0] 被原地修改成 11，外面看得到。

第 3 題 答案：(D)
- a.append(3) 是在原來的 list 上修改，外面的 a 變成 [1, 3]。
- b = b + [3] 產生一個新的 list 讓函式裡的 b 指過去，外面的 b 還是 [1]。

【錯】以為 b + [3] 也改到外面而選 2 2。

第 4 題 答案：(D)
- f 裡面的 x 是區域變數，怎麼改都不影響全域的 x。
- g 宣告了 global x，改到的是全域 x：5 × 2 = 10。

第 5 題 答案：(A)
- 往下走時（呼叫前）依序印 3、2、1，f(0) 直接返回。
- 回來時（呼叫後）從最裡層開始印：1、2、3。合起來 321123。

第 6 題 答案：(B)

- 呼叫順序：f(13) → f(6) → f(3) → f(1) → f(0)。
- 印的動作在呼叫之後，所以從最裡層 f(1) 開始印：1、1、0、1，正好是 13 的二進位 1101。

【錯】照呼叫順序印 13、6、3、1 的餘數 1、0、1、1，就會選錯成 1011。

第 7 題 答案：(B)

- f(1) 印出 1；f(2) = f(1)、印 2、f(1) → 121。
- f(3) = f(2)、印 3、f(2) → 121 3 121 = 1213121。

第 8 題 答案：(C)

- 設 C(n) 是呼叫 f(n) 的總次數：C(0) = C(1) = 1，C(n) = 1 + C(n-1) + C(n-2)。
- 由小算上去：C(2) = 3、C(3) = 5、C(4) = 9、C(5) = 15。

【錯】只數葉子或只數某一層，都會少算。每一個節點（包含 f(0)、f(1)）都要算。

第 9 題 答案：(A)

- 設 k(n) 是 fib(n) 裡 fib(2) 被呼叫的次數：k(2) = 1、k(3) = 1，k(n) = k(n-1) + k(n-2)。
- k(4) = 2、k(5) = 3、k(6) = 5。重複計算就是這樣越滾越多。

第 10 題 答案：(D)

- 呼叫依序是 g(48, 18) → g(18, 12) → g(12, 6) → g(6, 0)，每次先印 a：48 18 12 6。
- g(6, 0) 回傳 6，最外層再印出回傳值 6。

【錯】忘了最後還要印出回傳值，就會選 48 18 12 6。

第 11 題 答案：(A)

- 這個函式沒有終止條件：f(3) 呼叫 f(2)、f(1)、f(0)、f(-1)……永遠不會停。
- 超過 Python 的遞迴深度上限，出現 RecursionError。

【錯】以為「到 0 就會停」而選印出 6：程式裡根本沒有寫 n == 0 的判斷。

第 12 題 答案：(C)

- 遞迴本身寫得沒錯，但要疊 5000 層。
- Python 預設遞迴深度上限約 1000 層，超過就發生 RecursionError。

【錯】邏輯正確不代表一定跑得完；要用 sys.setrecursionlimit 放寬，或改寫成迴圈。

第 13 題 答案：(C)

- 搬 n 個 = 搬 n-1 個到中間 + 搬最大的 1 次 + 搬 n-1 個到目標，T(n) = 2T(n-1) + 1。
- T(1) = 1，所以 T(n) = 2^n - 1，T(10) = 1023。

第 14 題 答案：(D)

- 最大值是 9。呼叫的區間：[0,4]、[0,2]、[0,1]、[0,0]、[1,1]、[2,2]、[3,4]、[3,3]、[4,4]。
- 共 9 次。一般來說，分治切到剩一個，長度 n 會呼叫 2n - 1 次。

Part B 程式實作參考解答

實作 1 禮物包

解題思路

- 包數必須同時整除每一個 a_i，最多的包數就是所有數的最大公因數 g。
- 用遞迴的輾轉相除法：gcd(a, b) = gcd(b, a % b)，b = 0 時答案是 a。
- g 從 0 開始，一個一個做 g = gcd(g, a_i)（gcd(0, x) = x）；最後每包各有 a_i / g 顆。

【錯】只有一個數時，答案是它自己、每包 1 顆；別把 n = 1 當特例寫錯。

Python 禮物包

```python
import sys

def gcd(a, b):
    if b == 0:
        return a
    return gcd(b, a % b)

data = sys.stdin.read().split()
n = int(data[0])
a = list(map(int, data[1:1 + n]))
g = 0
for x in a:
    g = gcd(g, x)                # gcd(0, x) = x
print(g)
print(" ".join(str(x // g) for x in a))
```

時間複雜度：O(n log C)，C 是數值大小

實作 2 碎形尺

解題思路

- 題目的定義本身就是遞迴：ruler(n) = ruler(n-1)、記下 n、ruler(n-1)。
- 終止條件：n = 0 時什麼都不畫。
- 先把刻度存進陣列（串列），最後一次輸出，比較好控制空白。

【錯】刻度有 $2^n - 1$ 條，n = 17 時約 13 萬條；一邊遞迴一邊大量輸出時，記得用快速輸出。

【錯】把「記下 n」放在兩次遞迴的前面或後面，順序就完全不同了，寫完用 n = 3 對一次。

Python 碎形尺

```python
n = int(input())
marks = []

def ruler(k):
    if k == 0:
        return
    ruler(k - 1)
    marks.append(k)
    ruler(k - 1)

ruler(n)
print(" ".join(map(str, marks)))
```

時間複雜度：$O(2^n)$（要輸出的數字本來就有 $2^n - 1$ 個）

實作 3 費氏呼叫次數

解題思路

- 設 c(n) 是計算 fib(n) 的呼叫次數：c(0) = c(1) = 1（只有自己）。
- 其他情況：自己 1 次，加上 fib(n-1) 和 fib(n-2) 各自的呼叫次數：c(n) = 1 + c(n-1) + c(n-2)。
- fib(n) 和 c(n) 都用「記憶化」：算過就存起來，每個 n 只算一次。

【錯】直接照小華的寫法模擬來數次數，n = 85 要跑 8×10^{17} 次，永遠跑不完。

Python 費氏呼叫次數

```python
n = int(input())
memo = {}

def solve(k):                # 回傳 (fib(k), 呼叫次數)
    if k in memo:            # 算過就直接拿
        return memo[k]
    if k <= 1:
        ans = (k, 1)
    else:
        f1, c1 = solve(k - 1)
        f2, c2 = solve(k - 2)
        ans = (f1 + f2, 1 + c1 + c2)
    memo[k] = ans
    return ans

f, c = solve(n)
print(f, c)
```

時間複雜度：O(n)（記憶化後每個 n 只算一次）

實作 4 快速冪

解題思路

- 終止條件：b = 0 時答案是 1 % m（m = 1 時是 0）。
- 縮小問題：先算 half = $a^{b/2}$ mod m，只算一次。
- 組合答案：half × half mod m；b 是奇數再乘一次 a、再取餘數。深度只有約 60 層。

【錯】[x] 寫成 power(a, b/2) * power(a, b/2)：同一件事算兩次，呼叫次數又爆炸成 O(b)。

【錯】b = 0、m = 1 時答案是 0，不是 1：輸出前要記得 % m。

Python 快速冪

```python
def power(a, b, m):
    if b == 0:
        return 1 % m
    half = power(a, b // 2, m)      # 只算一次！
    r = half * half % m
    if b % 2 == 1:
        r = r * a % m
    return r

a, b, m = map(int, input().split())
print(power(a % m, b, m))
```

時間複雜度：O(log b)

實作 5 河內塔的第 k 步

解題思路

- 搬 n 個盤子（從 from 經過 via 到 to）分三段：前 $2^{n-1}-1$ 步把上面 n-1 個搬到 via；第 2^{n-1} 步把 n 號盤從 from 搬到 to；剩下的步數把 n-1 個從 via 搬到 to。
- 令 half = 2^{n-1}：k = half 就是答案；k < half 往第一段找；k > half 就往第三段找第 k - half 步。
- 每次只往其中一段走，深度最多 60 層，不用真的把 2^{60} 步都搬出來。

【錯】第一段的柱子是 (from, to, via)，第三段是 (via, from, to)，角色換錯答案就錯。

【錯】直接模擬把每一步都搬出來，n = 60 要 10^{18} 步，不可能跑完。

Python 河內塔的第 k 步

```python
def step(n, frm, via, to, k):
    half = 1 << (n - 1)
    if k == half:
        print(n, frm, to)
    elif k < half:
        step(n - 1, frm, to, via, k)         # 第一段
    else:
        step(n - 1, via, frm, to, k - half)  # 第三段

n, k = map(int, input().split())
step(n, 1, 2, 3, k)
```

時間複雜度：O(n)

自我診斷表

對完答案，數一數每個觀念錯幾題：全對圈綠、錯 1 題圈黃、錯 2 題以上圈紅。紅燈的觀念回去看講義那一段。

觀念	題號	錯幾題	燈號（圈一個）	回去看哪裡
函式與回傳值	A1		綠 黃 紅	講義第 1 段
傳值與傳參考	A2、A3		綠 黃 紅	講義第 2 段
區域與全域變數	A4		綠 黃 紅	講義第 3 段
遞迴三要素	A10、A11、B1		綠 黃 紅	講義第 4 段（階乘、輾轉相除）
遞迴追蹤	A5、A6、A7、B2		綠 黃 紅	講義第 5 段（print 在前或在後、遞迴樹）
呼叫次數	A8、A9、B3		綠 黃 紅	講義第 5、6 段（遞迴樹節點數、費氏數列爆炸）
河內塔、分治與遞迴深度	A12、A13、A14、B4、B5		綠 黃 紅	講義第 7 段

覺得有幫助？幫老師一個忙

- **訂閱** YouTube「莊老數學教室」：youtube.com/@chuanglaomath
- 看到老師的影片，順手**按個讚**
- 把這份免費講義**分享給同學**：wade-chuang.github.io/jenfoo/apcs（不用註冊，直接下載）